
Tutorial 11

1. The undirected Hamiltonian Cycle Problem is as follows: given an undirected graph G , is there a cycle which contains *all* the vertices? The undirected Hamiltonian Path problem is as follows: given an undirected graph G , is there a path which contains all the vertices? Show that the undirected Hamiltonian path is polynomial time reducible to the undirected Hamiltonian Cycle problem.

Sol. Let I be an input to the Hamiltonian path problem. Note that I consists of an undirected graph G . We need to produce a graph G' such that G has a Hamiltonian path if and only if G' has a Hamiltonian cycle. We proceed as follows: add a new vertex v to the graph G and add edges between v and every vertex in G - call this graph G' . Now if P is a Hamiltonian path in G starting at vertex s and ending at t , then v, s, P, t, v is a Hamiltonian cycle in G' . Conversely, if C is a Hamiltonian cycle in G' , then removing the vertex v from C gives a Hamiltonian path in G .

2. The Path Selection Problem can be defined as follows: given a directed graph G , a set of directed paths $P_1, \dots, P_r$ in the graph G , and a number k , is it possible to select at least k of these paths so that no two of the selected paths share any vertices? Prove that the Independent Set problem is polynomial time reducible to the Path Selection Problem.

Sol. Let $G = (V, E)$, k be an instance of the independent set problem. We will produce a directed graph H and paths $P_1, \dots, P_r$ in it such that G has an independent set of size k if and only if we can select k paths among $P_1, \dots, P_r$ which do not share any vertex. To do this, the vertex set of H has one vertex for every edge of G — we will denote these vertices by $e_1, \dots, e_m$, which are also the labels of the edges of G . Now we define the edges in H — between every pair of vertices in H there is a directed edge between them. Now we define n paths $P_1, \dots, P_n$ in G , one path P_i for each vertex v_i in G as follows. If the edges incident with v_i are $e_{i_1}, e_{i_2}, \dots, e_{i_k}$, we have a path $P_i : e_{i_1}, e_{i_2}, \dots, e_{i_k}$ in H .

Now suppose G has an independent set of size k . Let these vertices be $v_1, \dots, v_k$. Then the paths $P_1, \dots, P_k$ are disjoint because if P_i and P_j share a common vertex e in H , then e is incident with both v_i and v_j , but then these vertices cannot be part of an independent set in G . Conversely, suppose $P_1, \dots, P_k$ is a subset of k paths which do not share a vertex. Then the same argument shows that $v_1, \dots, v_k$ form an independent set.

3. The Subset-Sum Problem is defined as follows - Given a set of n positive integers $S = \{a_1, a_2, \dots, a_n\}$ and a target value W , the Subset Sum problem asks whether there exists a subset $T \subseteq S$ such that $\sum_{a_i \in T} a_i = W$.

Given a multiset of n positive integers $S = \{a_1, a_2, \dots, a_n\}$, the 2-Partition problem asks whether S can be partitioned into two subsets S_1 and S_2 such that $S_1 \cup S_2 = S$, $S_1 \cap S_2 = \emptyset$, and $\sum_{a_i \in S_1} a_i = \sum_{a_j \in S_2} a_j$.

Show that SUBSET-SUM is polynomial time reducible to 2-PARTITION.

Sol. Let $x_1, \dots, x_n, S$ be an instance of the subset sum problem — one needs to check if a subset of the numbers $x_1, \dots, x_n$ adds up to S exactly. We can always replace S by $\Sigma - S$ without changing the problem, where Σ denotes $\sum_i x_i$. Therefore, we can also assume that $S \leq \Sigma - S$, i.e., $\Sigma \geq 2S$. We now map it to an instance of the PARTITION problem. The set of numbers is $x_1, \dots, x_n, \Sigma - 2S$. The claim is that the subset sum instance has a solution if and only if PARTITION has a solution. Indeed, if there is a subset A of $\{x_1, \dots, x_n\}$ adding up to S , then the two parts: $A \cup \{\Sigma - 2S\}$, and the remaining numbers, both add up to $\Sigma - S$. Check the converse statement.

4. Define The Zero Weight Cycle problem as follows: you are given a directed graph $G = (V, E)$, with weights w_e on the edges $e \in E$. The weights can be negative or positive integers (or equal to 0). You need to decide if there is a cycle in G so that the sum of the edge weights on this cycle is exactly 0 (i.e., say YES if there is such a cycle, NO otherwise). Prove that the Subset Sum problem is polynomial time reducible to the Zero Weight Cycle Problem.

Sol. Consider an instance of the subset sum problem — numbers $x_1, \dots, x_n$ and a number S . We will produce a graph $G = (V, E)$ with edge weights such that G has a zero weight cycle if and only if the subset sum problem has a solution. For this, we introduce vertices $v_1, \dots, v_{n+1}$. Between v_i and v_{i+1} , we have an edge of cost x_i . Also, we have a vertex w_i and edges (v_i, w_i) and (w_i, v_{i+1}) of cost 0. Finally we have edge (v_{n+1}, v_1) of weight $-S$.

Now suppose the subset sum problem has a solution consisting of a subset T of $\{x_1, \dots, x_n\}$ which adds up to S . Now consider the following cycle: if $x_i \in T$, take the edge (v_i, v_{i+1}) , otherwise take the edges (v_i, w_i) and (w_i, v_{i+1}) . Further take the edge (v_{n+1}, v_1) . These add up to length 0. Conversely, suppose there is a zero weight cycle in G . Since (v_{n+1}, v_1) is the only edge with negative length, this cycle must contain this edge. Therefore there is a path from v_1 to v_{n+1} in this cycle for which the weights of edges add up to S . Now, consider those v_i for which this path takes the edge (v_i, v_{i+1}) . The corresponding numbers x_i add up to S .

5. We have seen the Interval Scheduling problem in class; here we consider a computationally much harder version of it that we will call Multiple Interval Scheduling (MIS). As before, you have a processor that is available to run jobs over some period of time. (E.g. 9 AM to 5 PM.)

People submit jobs to run on the processor; the processor can only work on one job at any single point in time. Jobs in this model, however, are more complicated than we've seen in the past: each job requires a set of intervals of time during which it needs to use the processor. Thus, for example, a single job could require the processor from 10 AM to 11 AM, and again from 2 PM to 3 PM. If you accept this job, it ties up your processor during those two hours, but you could still accept jobs that need any other time periods (including the hours from 11 to 2). Now, you're given a set of n jobs, each specified by a set of time intervals, and you want to answer the following question: For a given number k , is it possible to accept at least k of the jobs so that no two of the accepted jobs have any overlap in time?

Show that Independent Set is polynomial time reducible to MIS.

Sol. Take an instance of independent set: graph G and number k . We map it to an instance of the Multiple Interval Scheduling (MIS) problem as follows: the time interval stretches from 0 to m , where m is the number of edges. Let the edges be numbered $e_1, \dots, e_m$. Then the interval $[i, i + 1]$ will correspond to the edge e_i . For a vertex v , we define a job: this job will consist of intervals $[i, i + 1]$ for every edge e_i which has v as an end-point. It is easy to check

that G has an independent set of size k if and only if the corresponding k jobs in the MIS instance do not overlap (prove this yourself).